



Figure e Strumenti di Garanzia

Parla con chi può aiutarti:

➤ **Garante delle Studentesse e degli Studenti di Facoltà**

Ascolta segnalazioni su problemi con la didattica, esami, rapporti con docenti.

➤ **Referente DSA e Disabilità**

Tutela gli studenti con disabilità o DSA, garantendo loro partecipazione attiva.

➤ **Consigliera di Fiducia**

Fornisce consulenza e assistenza alle vittime di molestie sessuali.

➤ **Progetto SAFE ZONE**

Cerca il simbolo sulle porte degli Uffici per avere il sostegno su questioni legate alla violenza di genere.

➤ **Servizi Antiviolenza - Sapienza**

Centri di Orientamento e Servizi antiViolenza.

➤ **Commissioni GEP (Gender Equality Plan)**

Una per ciascuno dei sei nostri Dipartimenti, organizzano progetti e iniziative volti a diffondere la cultura della parità e delle pari opportunità.



SAFE ZONE



QUANDO VEDI QUESTO
SIMBOLO, SAI CHE
QUALCUNO TI POTRÀ
ASCOLTARE

FACOLTÀ DI SCIENZE
MATEMATICHE FISICHE
E NATURALI



SAPIENZA
UNIVERSITÀ DI ROMA

Esempio: 75 in base 2

Stesso algoritmo, più passi

divisione	quoziente	resto
75 / 2	37	1
37 / 2	18	1
18 / 2	9	0
9 / 2	4	1
4 / 2	2	0
2 / 2	1	0
1 / 2	0	1

risultato (resti letti dal basso)

1	0	0	1	0	1	1
64	32	16	8	4	2	1

verifica:

$$64 + 8 + 2 + 1 = 75$$

Base 16 (esadecimale)

Servono 16 numerali: dopo 9 si usano le lettere A ... F

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
										10	11	12	13	14	15

$$13_{10} = D_{16}$$

$$10_{16} = 16_{10} \quad 0 \times 16^0 + 1 \times 16^1$$

$$1F_{16} = 31_{10} \quad 16 + 15$$

$$20_{16} = 32_{10} \quad 2 \times 16$$

«10» vale

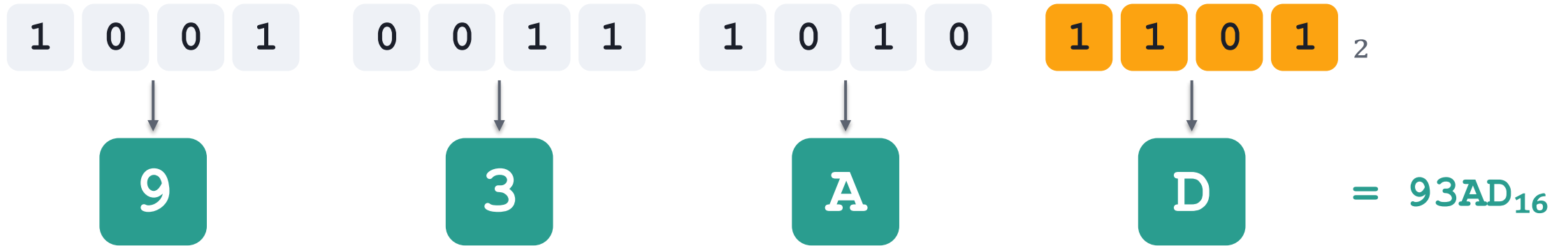
$$10_{10}=10 \quad 10_2=2 \quad 10_8=8 \quad 10_{16}=16$$

In C: prefisso 0x

```
int n = 0x1F; // vale 31
printf("%X", 31); // 1F
```

Da binario a esadecimale

$16 = 2^4$: ogni gruppo di 4 bit (da destra) è una cifra esadecimale



$$\begin{aligned} 1101 &= 1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 \\ &= 1 + 4 + 8 = 13_{10} = \text{D} \\ 1010 &= 10_{10} = \text{A} \end{aligned}$$

4 cifre in base 2

0000 ... 1111
0 ... 15
0 ... F

Base 8 (ottale)

Numerali 0 ... 7. $8 = 2^3$: si raggruppano i bit a 3 a 3, partendo da destra

0 1 2 3 4 5 6 7 $10_8 = 8_{10}$

1 0 1 1 0 1 ₂
↓ ↓
5 5
= 55₈

Prova tu

00101101₂ in base 8 e in
base 16

(gruppi di 3 per l'ottale, di 4 per l'esadecimale)

Se l'ultimo gruppo a sinistra è incompleto, si aggiungono zeri: non cambiano il valore.

Esercizi

Prima con carta e penna, poi scrivendo un programma in C che faccia le divisioni successive

1 Base 2

Converti 1073_{10} in base 2 con le divisioni successive.

2 Basi 8 e 16

Scrivi 00101101_2 in base 8 e in base 16.

3 Base 10

Quanto vale $93AD_{16}$ in base 10?

4 Combinazioni

Quanti numeri si scrivono con 8 bit? E con 2 cifre esadecimali? Perché sono uguali?

Riepilogo

1

Compilatore

traduce il sorgente (C) in istruzioni numeriche per la macchina

2

Posizionale

$N = \sum a_n b^n$; con c cifre si scrivono b^c numeri

3

Base 2

divisioni successive per 2, resti letti dal basso

4

Basi 8 e 16

gruppi di 3 o 4 bit; in C si scrive 0x1F

0 1 1 0 1

Rappresentazione dei numeri nel calcolatore

Bit di segno, complemento, base 2 e numeri reali (IEEE 754)

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027



SAPIENZA
UNIVERSITÀ DI ROMA

Numeri con segno: 4 bit

Il primo bit indica il segno (0 = +, 1 = -), gli altri 3 il valore



segno usati per il numero

Senza segno: 0000 ... 1111 = 0 ... 15

Con segno: 0 → +, 1 → -

0000 e 1000 rappresentano entrambi lo zero:
sono ridondanti.

0	0	0	0	0		1	0	0	0	-0
0	0	0	1	1		1	0	0	1	-1
0	0	1	0	2		1	0	1	0	-2
⋮						⋮				
0	1	1	1	7		1	1	1	1	-7

Quanti numeri distinti?

Il conto per 4 bit con bit di segno

$$2^4 = 16$$

combinazioni

2

ridondanti per lo 0

15

numeri distinti: -7 ... +7

Problema: due zeri e regole diverse per somma e sottrazione.

Serve una rappresentazione in cui $x + (-x) = 0$ funzioni «da sola».

Rappresentazione a complemento

Vogliamo che la sottrazione diventi una somma

$$x = 13$$

$$y = -13$$

$$13 - 13 = 0$$

$$13 + (-13) = 0$$

Cerchiamo un numero K (il complemento di x) che, sommato a x, dia zero usando solo un numero fisso di cifre.

Allora K rappresenta $-x$.

Quanti numeri con n cifre?

Esempio: 4 cifre in base 10

0 0 0 0

più piccolo

9 9 9 9

più grande

numero di combinazioni

base^{n. cifre}

$$10^4 = 10\ 000$$

Complemento in base 10

Il complemento K di x si definisce da $x + K = \text{base}^n$

$$x = 3547$$

$$K = 10^4 - x$$

$$K = 10000 - 3547 = 6453$$

	3	5	4	7
+	6	4	5	3
1	0	0	0	0

la 5ª cifra non ci sta: si perde

Con 4 cifre $3547 + 6453 = 0000$, come $3547 - 3547$.
Quindi -3547 è rappresentato da 6453.

È così che funziona un registro

Il registro della CPU ha un numero fisso di cifre: il riporto in più si perde. Proprio per questo il complemento funziona.

Metà positivi, metà negativi

4 cifre in base 10: come si dividono le 10 000 combinazioni

0000 ... 4999

rappresentano 0 ... +4999

5000 ... 9999

rappresentano -5000 ... -1

5000

↔ -5000

5001

↔ -4999

6453

↔ -3547

9999

↔ -1

Con 10 cifre: 0 ... 4 999 999 999 positivi, da $5 \cdot 10^9$ (= -5 000 000 000) a 9 999 999 999 (= -1) negativi.

Base 2: bit e byte

Il calcolatore usa solo due cifre: 0 e 1



ogni casella è 1 bit (vale 0 oppure 1)

$1024 = 2^{10}$: in informatica i multipli seguono le potenze di 2.

1 byte

= 8 bit

1 KB

= 1024 byte

1 MB

= 1024 KB

Complemento in base 2

$b = 2, n = 3$ cifre: $2^3 = 8$ combinazioni, 000 ... 111

Senza negativi: 0 ... 7 ($= 111_2$)

$$x = 011_2 = 3$$

$$x + K = b^n = 1000_2$$

$$K = 1000 - 011 = 101$$

sottrazione in colonna (con prestiti)

$$\begin{array}{r} 1000 \\ - 011 \\ \hline 101 \end{array}$$

011 ↔ +3

101 ↔ -3

verifica: $011 + 101 = 1000 \rightarrow 000$

La sottrazione con il prestito

$1000_2 - 011_2$ in 3 passaggi: come in base 10, ma il prestito vale 2 ($= 10_2$)

all'inizio	1	0	0	0	
passo 1	0	10	0	0	l'1 presta: lo 0 accanto diventa 10
passo 2	0	1	10	0	10 presta 1: resta 1, il successivo diventa 10
passo 3	0	1	1	10	ancora: resta 1, l'ultima cifra diventa 10

ora si sottrae cifra per cifra

	0	1	1	10
—	0	0	1	1
	0	1	0	1
	0-0	1-0	1-1	10-1

In base 10 è la stessa cosa: $1000 - 3$: l'1 presta, gli zeri diventano 10 e poi 9 $\rightarrow 09910 \rightarrow 997$

Tutti i numeri a 3 bit

Il primo bit fa da segno: 0 → positivo, 1 → negativo

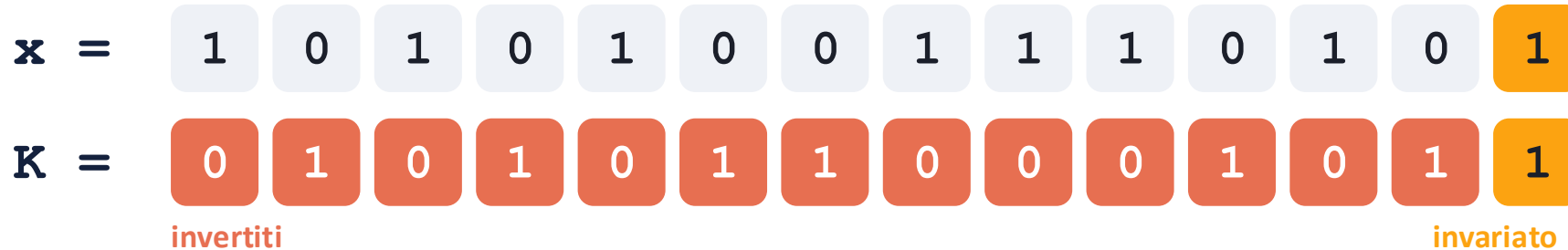
0	0	0	0	1	0	0	-4
0	0	1	1	1	0	1	-3
0	1	0	2	1	1	0	-2
0	1	1	3	1	1	1	-1

Da ricordare

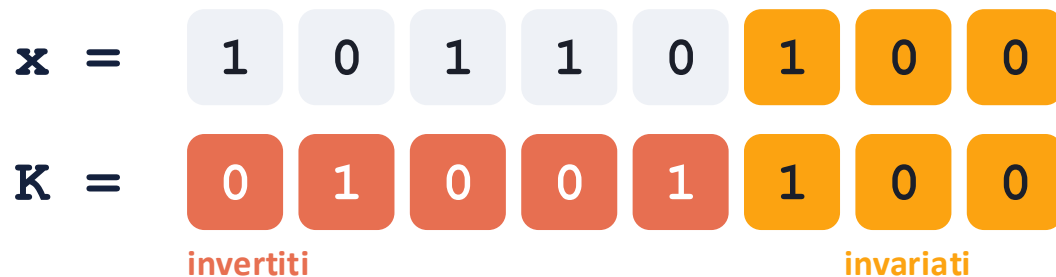
- Un solo zero: 000
- Intervallo: da -4 a +3
- Con n bit: da -2^{n-1} a $2^{n-1} - 1$
- -1 è sempre «tutti 1»

Calcolare il complemento: metodo 1

Da destra lascio i bit invariati fino al primo 1 compreso, poi scambio $0 \leftrightarrow 1$



il primo 1 da destra è l'ultima cifra



il primo 1 da destra è la terza cifra: 100 resta uguale
 $10110100 + 01001100$
 $= 1\ 00000000 \rightarrow 00000000$

Prova tu: quanto fa $K + x$ nell'esempio 1?

Suggerimento: con 14 bit il risultato deve essere 2^{14} , cioè 0 una volta perso il riporto.

Calcolare il complemento: metodo 2

Complemento a 1 (inverto tutti i bit) + somma di 1

x	1	0	1	0	numero di partenza
x'	0	1	0	1	complemento a 1: $0 \leftrightarrow 1$
x' + 1	0	1	1	0	complemento a 2 = K

in base 2

$$1 + 1 = 10$$

scrivo 0 e riporto 1, come $5 + 5 = 10$ in base 10

Verifica: $1010 + 0110 = 10000 \rightarrow 0000$

Interi su una macchina a 32 bit

Un intero occupa 32 bit, rappresentato a complemento

minimo

-2^{31}

-2 147 483 648

massimo

$2^{31} - 1$

2 147 483 647

2^{32} combinazioni: metà negativi ($-2^{32}/2 \dots -1$), metà zero e positivi ($0 \dots 2^{32}/2 - 1$).

In C: `INT_MIN` e `INT_MAX` in `<limits.h>`

Interi su una macchina a 64 bit

Le macchine moderne (PC, Mac, server) hanno registri a 64 bit

minimo

-2^{63}

-9 223 372 036 854 775 808

massimo

$2^{63} - 1$

9 223 372 036 854 775 807

$\approx 9.2 \times 10^{18}$: nove miliardi di miliardi.

$2^{32} \approx 4$ miliardi di volte più che con 32 bit.

Attenzione: anche a 64 bit `int` resta di 32 bit. Per 64 bit: `long long`, con `LLONG_MIN` e `LLONG_MAX`.



Terminale — Mac arm64

```
$ ./sizeof
int           : 4 byte
long          : 8 byte
long long     : 8 byte
puntatore    : 8 byte
```

Numeri reali: sistema posizionale

Ogni cifra pesa una potenza della base, anche dopo la virgola

base 10

$$23.20197 = 2 \times 10^1 + 3 \times 10^0 + 2 \times 10^{-1} + 0 \times 10^{-2} + \dots + 7 \times 10^{-5}$$

base 2

$$\begin{aligned} 101.101_2 &= 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ &= 4 + 1 + 0.5 + 0.125 = 5.625 \end{aligned}$$

Rappresentazione normale

Una sola cifra (non nulla) prima della virgola

$$x = (\text{segno}) m \times b^e$$

m mantissa, $1 \leq m < b$

b base

e esponente

$$23.20197 = 2.320197 \times 10^1 \quad m = 2.320197, e = 1$$

$$-127.23 = -1.2723 \times 10^2 \quad m = 1.2723, e = 2$$

Non normale: 232.0197×10^{-1} rappresenta lo stesso numero, ma la forma non è unica.

Numero razionale nello standard IEEE 754

Singola precisione (float): 32 bit



$$x = (-1)^s \times m \times 2^e$$

Mantissa: $m = 1 + b_1 \times 2^{-1} + b_2 \times 2^{-2} + \dots + b_{23} \times 2^{-23}$

L'1 prima della virgola è sottinteso: in base 2 la forma normale inizia sempre con 1.

L'esponente su 8 bit

$2^8 = 256$ combinazioni: servono esponenti sia positivi sia negativi

$$2^5 = 32$$

$$2^6 = 64$$

$$2^7 = 128$$

$$2^8 = 256$$

valori memorizzati E (senza segno)

$$00000000 \dots 11111111 = 0 \dots 255$$

esponente vero: $e = E - 127$ (offset, «bias»)

$$E = 127 \rightarrow e = 0$$

$$E = 154 \rightarrow e = 27$$

0 e 255 sono riservati (zero, infinito, NaN)

Esempio: $x = 1.234 \times 2^{27} \rightarrow E = 27 + 127 = 154 = 10011010_2$

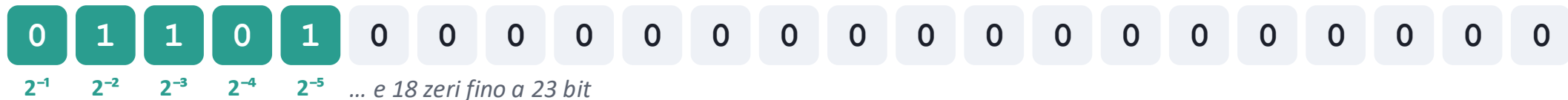
La mantissa nei 23 bit

Esempio: $x = 5.625$, lo stesso numero della slide 16

in base 2 $5.625 = 101.101_2$

forma normale $= 1.01101_2 \times 2^2$ ($e = 2$)

si scrive solo ciò che viene dopo la virgola: **01101**



L'1 prima della virgola non si scrive: in forma normale c'è sempre, quindi è sottinteso («bit implicito»).

verifica:

$$1.01101_2 = 1 + 1/4 + 1/8 + 1/32$$
$$= 1.40625$$

$$1.40625 \times 2^2 = 5.625 \checkmark$$

Tutti i 32 bit: $x = -5.625$

Segno, esponente e mantissa, uno dopo l'altro



segno

$x < 0$
→ $s = 1$

esponente

$e = 2$
 $E = 2 + 127 = 129$
 $= 10000001$

mantissa

1.01101
→ 01101
e poi 18 zeri



Terminale

```
$ ./bit_float
x = -5.625000
bit: 1 10000001 011010000000000000000000
esadecimale: C0B40000
```

in esadecimale

C0B40000

4 bit = 1 cifra

Esercizi

Da fare a casa, prima con carta e penna, poi controllando con un programma in C

1 Complemento

Calcola $K + x$ per $x = 10101001110101$ (14 bit).

2 Forma normale

Scrivi in forma normale, in base 10: $x = 127.93$ e $x = -975.75$

3 IEEE 754

Scrivi -975.75 come (segno) $m \times 2^e$ in base 2, e poi i suoi 32 bit.

4 Intervalli

Con 8 bit: quanti numeri distinti con il bit di segno? E quali interi a complemento?

Riepilogo

1

Bit di segno

semplice, ma lo zero ha due rappresentazioni: 0000 e 1000

2

Complemento

$K = b^n - x$ rappresenta $-x$: la sottrazione diventa una somma

3

Base 2

Metodo rapido: inverti i bit e sommo 1

4

Interi n bit

da -2^{n-1} a $2^{n-1} - 1$; 32 bit: $\pm 2.1 \cdot 10^9$, 64 bit: $\pm 9.2 \cdot 10^{18}$

5

Reali

$(-1)^s \times m \times 2^e$: 1 bit segno, 8 esponente, 23 mantissa